



Cadastro de Funcionários

Especificação Técnica de Projeto

Desenvolvimento de uma aplicação web de cadastro de funcionários com consumo de APIs externas, utilizando apenas **HTML, CSS e JavaScript puro**.

ATIVIDADE PRÁTICA

FRONT-END

APIS REST

Visão Geral da Atividade

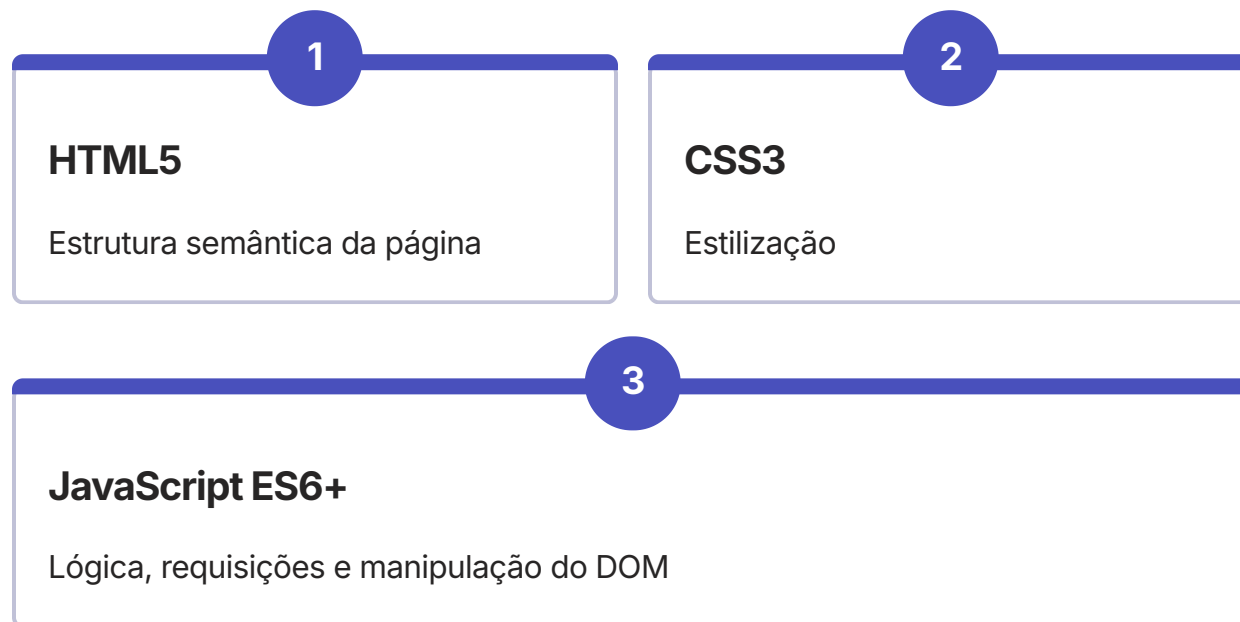
⚠ Essa é uma atividade **obrigatória** para que sua presença na disciplina de Lógica de Programação do dia **19/06/26** seja validada.

O objetivo é construir uma página web que simula um sistema real de **Cadastro de Funcionários**.

A aplicação deve consumir dados de **duas APIs simultaneamente**, cruzando informações para exibir um card completo e visualmente consistente.

O resultado final deve ser uma interface limpa e funcional, reproduzindo fielmente o **layout de referência** fornecido pelo professor.

Tecnologias Obrigatórias



ⓘ Nenhum framework ou biblioteca externa é permitido. Todo o código deve ser escrito em HTML, CSS e JS puros.

Guia de Implementação e Entrega

Para garantir o sucesso e a correta avaliação do seu projeto, siga atentamente as diretrizes de configuração, desenvolvimento e entrega.

Configuração Inicial

- Nome da pasta do projeto: `cadastro-funcionario` (minúsculas, sem acentos ou espaços).
- Estrutura de arquivos: `index.html`, `style.css` e `script.js`.

Fluxo de Commits

- Commit inicial obrigatório: `"inicio do projeto"` (para registrar o início da atividade).
- Commit final obrigatório: `"projeto finalizado"` (para registrar a conclusão da atividade).
- Evolução do código: Faça commits a cada etapa finalizada.
Exemplos: `"html finalizado"`, `"estilo concluído"`, `"função de fetch concluída"`, `"função de busca por id finalizada"`.

Requisitos de Entrega

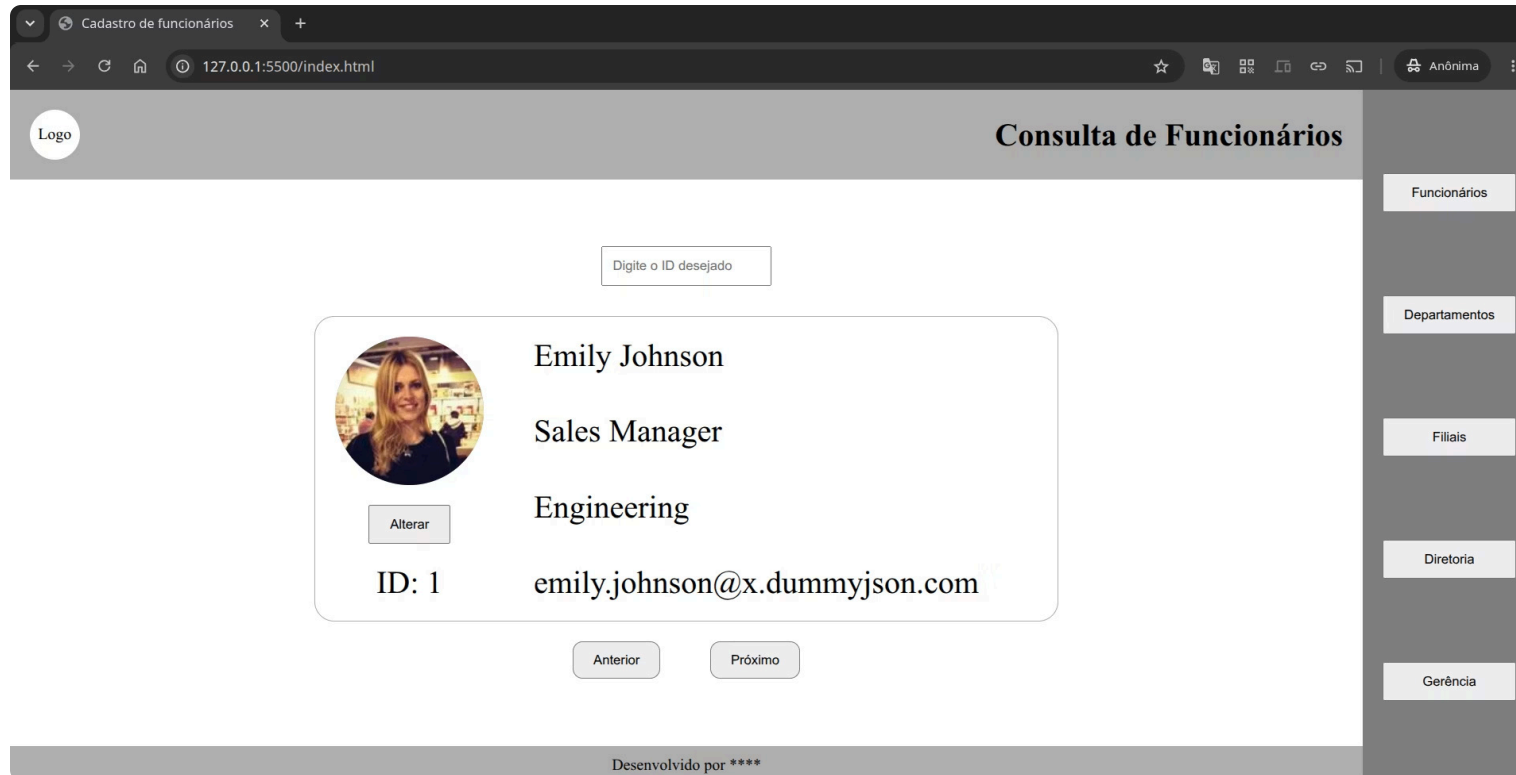
- Repositório GitHub: Deve ser **público** para acesso e avaliação.
- Publicação: O projeto final deve estar hospedado e acessível via **GitHub Pages**.
- A entrega só será aceita no dia **19/06/2026, até às 23:59h**, com envio do link do repositório para o e-mail: tiago.guimaraes@sp.senai.br

 A evolução do seu código será avaliada por meio do histórico de commits. Mantenha um registro claro e consistente do seu progresso.

 **Importante:** A atividade deve ser iniciada (com o commit inicial: ***"inicio do projeto"***) e finalizada (com o commit final: ***"projeto finalizado"***) dentro do **dia 19/06/2026 (sexta-feira)**, para que sua presença seja validada.

Estrutura do Layout

O layout deve seguir rigorosamente a imagem de referência, com os seguintes blocos estruturais:



 Cada zona possui responsabilidades visuais bem definidas.

A **fidelidade ao layout de referência** é um critério direto de avaliação.

 **Atenção:** Apenas o campo de busca por **ID** e os botões **Anterior** e **Próximo** são funcionais. Os botões laterais são apenas para representação do layout e não possuem função real.

O Card do Funcionário

O card é o elemento central da interface. Deve possuir **bordas arredondadas**, contorno fino e fundo branco, dividido em dois lados:

Lado Esquerdo

- Foto de perfil **circular** do funcionário
- Botão "**Alterar**" abaixo da foto
- Texto "**ID: [número]**" abaixo do botão

Lado Direito

- **Nome completo** em destaque (fonte maior)
- **Cargo** do funcionário
- **Departamento**
- **E-mail** do funcionário

- ❏ Todos os campos do card são preenchidos dinamicamente via JavaScript, cruzando dados das duas APIs.



Emily Johnson

Cargo: Sales Manager

Depto: Engineering

Alterar

ID: 1

emily.johnson@x.dummyjson.com

Requisitos de Layout e Design



Layout

Siga o layout fornecido, ou seja: interface limpa, sem poluição visual.



Cabeçalho

Logo circular à esquerda, título centralizado "**Cadastro de Funcionários**" e campo de busca à direita com placeholder descritivo.



Barra Lateral

5 botões empilhados verticalmente: **Funcionários, Departamentos, Filiais, Diretoria e Gerência**. Espaçamento igual entre os itens.

APIs Utilizadas

API 1 — DADOS DO FUNCIONÁRIO

DummyJSON Users

<https://dummyjson.com/users/>

Fornece os dados cadastrais completos do funcionário. Campos a extrair:

- **Nome** completo
- **Cargo** (title)
- **Departamento**
- **E-mail**
- **ID e gênero** (usado para filtrar a segunda API)

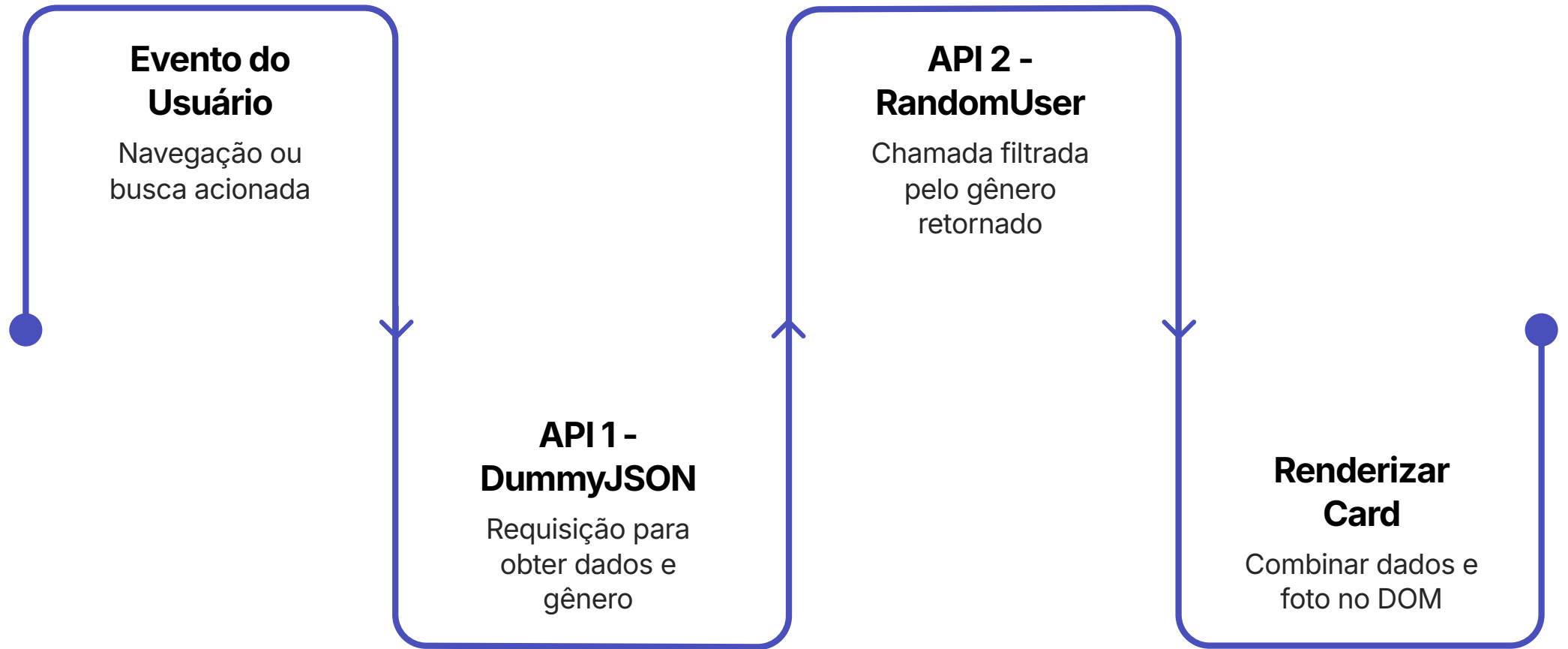
API 2 — FOTO DE PERFIL

Random User API

<https://randomuser.me/api/?gender=>

Fornece a foto de perfil aleatória. O gênero recebido da API 1 (male ou female) é passado como filtro para garantir **coerência visual** com o cadastro. Utilize sempre a imagem de **maior definição** disponível.

Fluxo de Consumo das APIs



O encadeamento correto das duas requisições é um ponto crítico de avaliação. O gênero obtido na primeira chamada **deve ser utilizado como parâmetro** na segunda chamada, garantindo que a foto exibida corresponda ao perfil do funcionário.

Comportamento dos Botões e Interações

1

Botão "Próximo"

Incrementa o ID atual em +1 e realiza uma nova chamada às duas APIs, atualizando todos os campos do card dinamicamente.

2

Botão "Anterior"

Decrementa o ID em -1. Deve **tratar o limite inferior** (ID mínimo = 1), impedindo que o valor chegue a zero ou negativo.

3

Campo de Busca

Ao pressionar **Enter**, realiza busca por ID numérico.

4

Carregamento Inicial

Ao abrir a página, o sistema deve automaticamente carregar e exibir os dados do **funcionário de ID 1** sem nenhuma interação do usuário.

Critérios de Avaliação

A nota será atribuída com base em **dois pilares principais**: fidelidade ao layout de referência e funcionamento completo da aplicação.

Layout e Design (30%)

- Estrutura idêntica à imagem de referência
- Card com os dois lados corretamente posicionados

Funcionalidade (70%)

- Consumo correto das duas APIs
- Cruzamento de dados por gênero para a foto
- Navegação Próximo/Anterior funcionando
- Busca por ID corretamente implementada



Checklist Final de Entrega

Antes de submeter o projeto, verifique se todos os itens abaixo foram implementados corretamente:

Estrutura e Visual

- ☐ Cabeçalho com logo, título e campo de busca
- ☐ Barra lateral com 5 botões
- ☐ Card centralizado com foto circular
- ☐ Botão "Alterar" e texto do ID no card
- ☐ Nome, cargo, departamento e e-mail no card
- ☐ Botões "Anterior" e "Próximo" posicionados corretamente
- ☐ Rodapé com nome do desenvolvedor

Lógica e Funcionamento

- ☐ Carregamento automático do ID 1 ao abrir a página
- ☐ Requisição à API DummyJSON por ID
- ☐ Requisição à API RandomUser com filtro de gênero
- ☐ Foto de maior definição sendo exibida
- ☐ Navegação por Próximo e Anterior
- ☐ Busca por ID ao pressionar Enter
- ☐ Código limpo, *identado* e comentado onde for necessário

⚠ O projeto será avaliado diretamente no **GitHub**. Certifique-se de que não há erros no console e que todas as funcionalidades operam corretamente antes da entrega. **Lembre-se:** É obrigatório a publicação no **GitHub Pages** o envio do link do **repositório do projeto**, com os **commits** para acompanhamento da evolução da execução da atividade.